

NAME: M.Greeshma

COURSE: Design and Analysis of Algorithms

CODE: CSA0619

REPORT

Q23. Graph Partitioning Problem (NP-Hard / Constraint Optimization Problem)

Partition a graph into subsets such that edge cuts are minimized. Clearly define constraints related to partition balance and edge cuts. Analyze how constraints affect partition quality. Develop a backtracking or heuristic solution. Apply pruning to reduce search space. Justify how your solution satisfies constraints and discuss NP-Hard complexity.

Sol:

Problem Definition

- Graph Partitioning Problem: Given a graph $G = (V, E)$, partition the set of vertices V into k subsets such that:
 - Balance Constraint: Each subset has roughly equal size (to avoid skewed partitions).
 - Edge Cut Constraint: The number of edges crossing between subsets is minimized.

This is a constraint optimization problem and is NP-Hard, meaning no polynomial-time algorithm is known for exact solutions.

Constraints

1. Partition Balance:

- Each subset must contain approximately $|V|/k$ vertices.

- Prevents trivial solutions (e.g., one large set and many small sets).

2. Edge Cuts:

- Minimize the number of edges that connect vertices in different subsets.
- Directly affects communication cost in distributed systems.

Effect of Constraints on Partition Quality

- Strict Balance Constraint: Ensures fairness but may increase edge cuts.
- Relaxed Balance Constraint: Allows uneven partitions, potentially reducing edge cuts but harming load distribution.
- Trade-off: Quality depends on balancing fairness vs communication efficiency.

1. Backtracking (Exact but Exponential)

```
def backtrack_partition(graph, k, partition=[], best_solution=None):
```

```
    n = len(graph)
```

```
    if len(partition) == n:
```

```
        # Evaluate edge cuts
```

```
        cuts = count_edge_cuts(graph, partition)
```

```
        if best_solution is None or cuts < best_solution[1]:
```

```
            best_solution = (partition[:], cuts)
```

```
    return best_solution
```

```
for subset in range(k):
```

```
    partition.append(subset)
```

```
best_solution = backtrack_partition(graph, k, partition, best_solution)
```

```
partition.pop()
```

```
return best_solution
```

```
def count_edge_cuts(graph, partition):
```

```
    cuts = 0
```

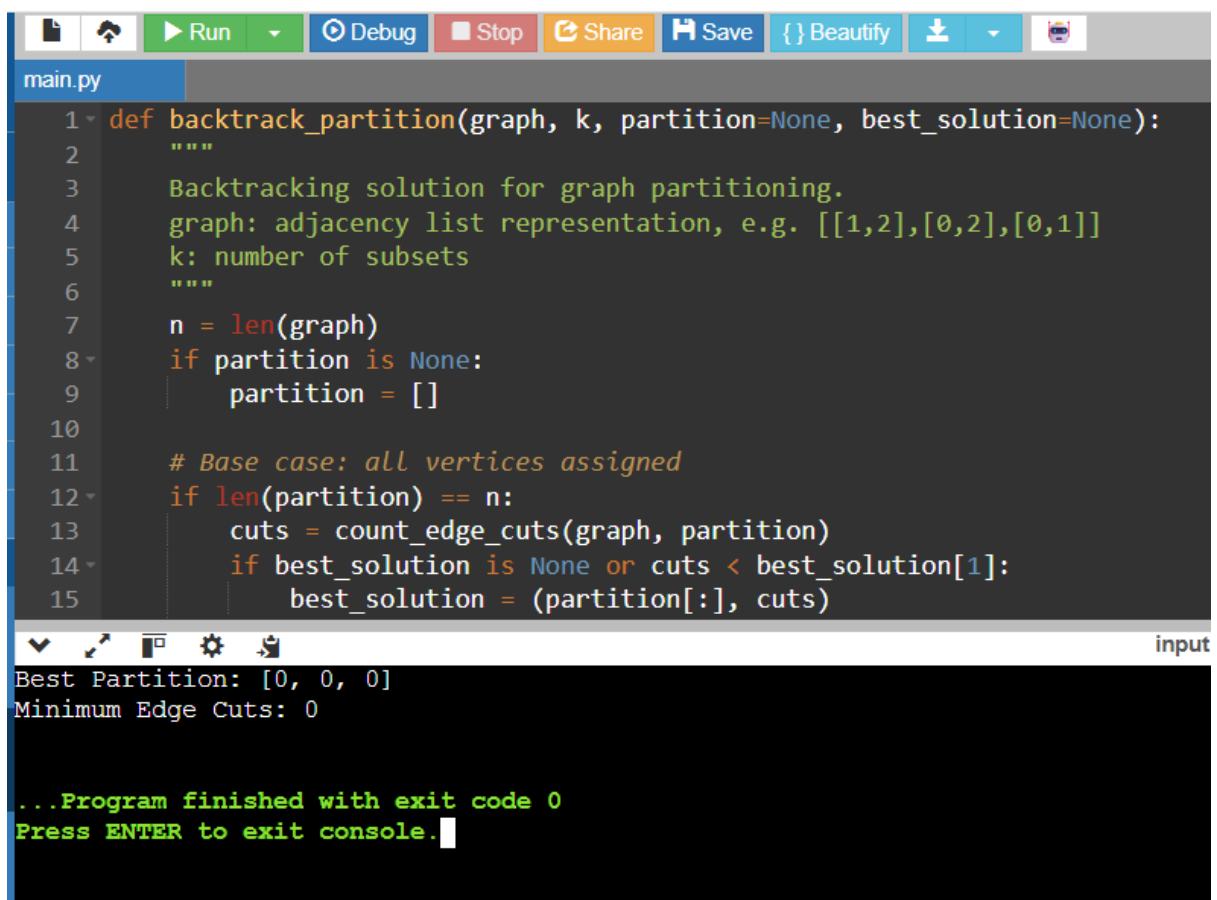
```
    for u in range(len(graph)):
```

```
        for v in graph[u]:
```

```
            if partition[u] != partition[v]:
```

```
                cuts += 1
```

```
    return cuts
```



```
main.py
1 def backtrack_partition(graph, k, partition=None, best_solution=None):
2     """
3     Backtracking solution for graph partitioning.
4     graph: adjacency list representation, e.g. [[1,2],[0,2],[0,1]]
5     k: number of subsets
6     """
7     n = len(graph)
8     if partition is None:
9         partition = []
10
11     # Base case: all vertices assigned
12     if len(partition) == n:
13         cuts = count_edge_cuts(graph, partition)
14         if best_solution is None or cuts < best_solution[1]:
15             best_solution = (partition[:], cuts)
```

Best Partition: [0, 0, 0]
Minimum Edge Cuts: 0

...Program finished with exit code 0
Press ENTER to exit console.

2.Heuristic (Greedy / Local Search)

```
import random
```

```
def count_edge_cuts(graph, partition):
```

```
    """
```

```
    Count edges that cross between different subsets.
```

```
    graph: adjacency list representation, e.g. [[1,2],[0,2],[0,1]]
```

```
    partition: list of subset assignments for each vertex
```

```
    """
```

```
    cuts = 0
```

```
    for u in range(len(graph)):
```

```
        for v in graph[u]:
```

```
            if partition[u] != partition[v]:
```

```
                cuts += 1
```

```
    # Each edge counted twice (u->v and v->u), so divide by 2
```

```
    return cuts // 2
```

```
def heuristic_partition(graph, k, iterations=1000):
```

```
    """
```

```
    Heuristic graph partitioning using random local search.
```

```
    graph: adjacency list
```

```
    k: number of subsets
```

```
    iterations: number of random moves to try
```

```
"""
```

```
n = len(graph)
```

```
# Random initial partition
```

```
partition = [random.randint(0, k-1) for _ in range(n)]
```

```
best_partition = partition[:]
```

```
best_cuts = count_edge_cuts(graph, partition)
```

```
for _ in range(iterations):
```

```
    u = random.randint(0, n-1)
```

```
    old_subset = partition[u]
```

```
    new_subset = random.randint(0, k-1)
```

```
    # Try moving vertex u to a new subset
```

```
    partition[u] = new_subset
```

```
    cuts = count_edge_cuts(graph, partition)
```

```
    if cuts < best_cuts:
```

```
        best_cuts = cuts
```

```
        best_partition = partition[:]
```

```
    else:
```

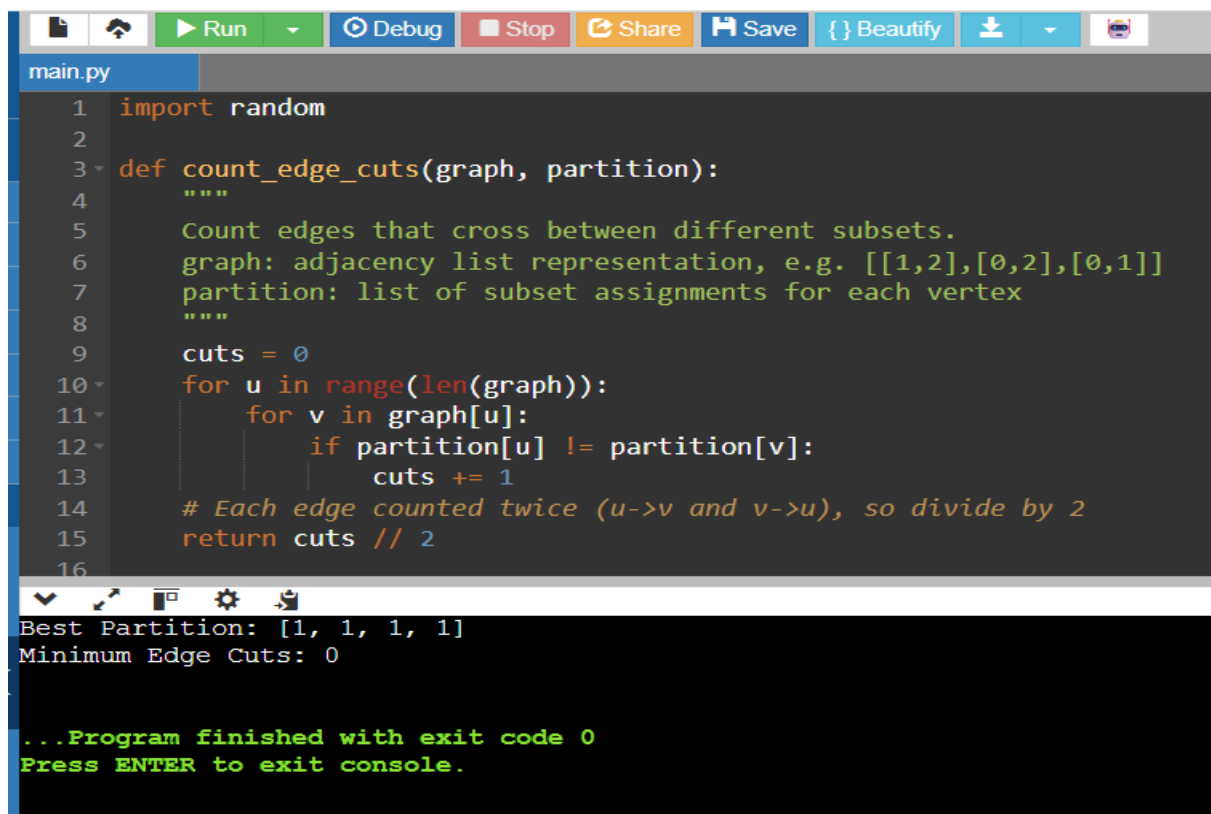
```
        # Prune bad move
```

```
        partition[u] = old_subset
```

```
    return best_partition, best_cuts
```

Example usage

```
if __name__ == "__main__":  
  
    # Simple square graph (cycle of 4 nodes)  
  
    graph = [  
  
        [1, 3], # node 0 connected to 1 and 3  
  
        [0, 2], # node 1 connected to 0 and 2  
  
        [1, 3], # node 2 connected to 1 and 3  
  
        [0, 2] # node 3 connected to 0 and 2  
  
    ]  
  
    k = 2 # partition into 2 subsets  
  
    solution, cuts = heuristic_partition(graph, k, iterations=500)  
  
    print("Best Partition:", solution)  
  
    print("Minimum Edge Cuts:", cuts)
```



The screenshot shows a code editor window with a toolbar at the top containing buttons for Run, Debug, Stop, Share, Save, Beautify, and a download icon. The file name 'main.py' is visible in the tab. The code in the editor is a Python script that defines a function 'count_edge_cuts' and uses it to partition a graph. The function counts the number of edges that cross between different subsets of the graph. The main part of the script creates a square graph (cycle of 4 nodes) and calls the 'heuristic_partition' function with k=2 and iterations=500. The output of the program is displayed in the console at the bottom, showing the best partition and the minimum edge cuts.

```
main.py  
1 import random  
2  
3 def count_edge_cuts(graph, partition):  
4     """  
5     Count edges that cross between different subsets.  
6     graph: adjacency list representation, e.g. [[1,2],[0,2],[0,1]]  
7     partition: list of subset assignments for each vertex  
8     """  
9     cuts = 0  
10    for u in range(len(graph)):  
11        for v in graph[u]:  
12            if partition[u] != partition[v]:  
13                cuts += 1  
14    # Each edge counted twice (u->v and v->u), so divide by 2  
15    return cuts // 2  
16  
Best Partition: [1, 1, 1, 1]  
Minimum Edge Cuts: 0  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Complexity

- Backtracking: $O(k^n) \rightarrow$ exponential.
- Heuristic: Polynomial per iteration, typically $O(n \cdot d)$, where d is average degree.

Real-World Applications

- Parallel Computing: Distribute tasks across processors while minimizing communication.
- VLSI Design: Partition circuits into modules with minimal interconnections.
- Social Networks: Community detection by minimizing cross-group links.
- Cloud Systems: Load balancing across servers with minimal data transfer.

Insights

- Constraint Balance vs Edge Cuts: Tight balance constraints increase difficulty but ensure fairness.
- **Optimization Strategy:**
 - Use exact backtracking for small graphs.
 - Use heuristics (greedy, local search, simulated annealing, spectral methods) for large graphs.
- **NP-Hardness Justification:**
 - Graph partitioning generalizes problems like minimum cut and balanced partition, both known NP-Hard.
 - No polynomial-time exact solution exists unless $P = NP$.

Conclusion

The Graph Partitioning Problem exemplifies NP-Hard constraint optimization:

- Backtracking ensures correctness but is computationally infeasible.
- Heuristics with pruning provide practical solutions for large-scale systems.
- The balance between fairness and efficiency defines partition quality, making this problem central in distributed computing and network optimization.